First Report

Group number	69	
member 1	QM no: 231225959	Name: Dongting Ge
member 2	QM no: 231225926	Name: Jie Gao
member 3	QM no: 231225753	Name: Yihang Dong
member 4	QM no: 231225948	Name: Yibo Wang
member 5	QM no: 231225845	Name: Kerui Wang
member 6	QM no: 231225937	Name: Yanzhou Wang

1. Requirements

1.1 Project context overview

The project brief defines a Teaching Assistant recruitment system for **three core actors**: **TA applicant, Module Organiser (MO) and administrator.** Our first task was to convert these abstract roles into a realistic workflow while respecting the coursework constraints: the software must be a stand-alone Java application or a lightweight Servlet/JSP system; data must be stored in simple text formats rather than a database; and the final product should demonstrate selected core functions through incremental Agile delivery. Because the team must submit a backlog, a complete prototype and a short report before full implementation, stage one was treated as a requirements-engineering exercise that had to connect every artefact: the brief defines the business problem, the stories translate it into user value, the prototype turns it into concrete interaction flow, and the backlog prepares it for incremental delivery.

Based on the uploaded handout, our product backlog and low-fidelity prototype, we defined the workflow as several steps: (1) a TA builds a structured profile and uploads a CV, (2) the TA browses vacancies and submits an application, (3) the MO publishes jobs, reviews applicants and updates status, and (4) the administrator monitors recruitment progress and manages exported data/logs. Advanced filtering, abnormal-account handling and AI matching were deliberately deferred because they improve efficiency but not necessary needed.

Users	Main need addressed	Representative backlog/prototype evidence
TA applicant	Manage profile data once, upload CV, apply efficiently and track feedback.	TA001–TA005; prototype pages 2–5.
MO	Publish positions, review applicants and update decisions in one place.	MO001–MO003; prototype pages 8–11.
Admin	See recruitment status globally, export records and follow up unfilled vacancies.	AD002–AD004; prototype pages 6–8.

1.2 Fact-finding techniques used

Technique	How it was applied	Why it was suitable	Main outputs

Background reading	Studied the course handout and report template to extract actors (TA/MO/Admin), delivery phases, and technical constraints.	Established project boundaries quickly to ensure development aligned with marking expectations.	Actor list, technical restrictions, and initial scope definition.
Interviewing	Conducted asynchronous team discussions and role-played as TA, MO, and Admin to align on requirement details; we also asked out TA to join in our discussion	Compensated for the lack of external stakeholders; helped uncover pain points like TA workload caps.	Core functional needs and priorities across all three roles.
Observation	Reviewed the low-fidelity prototype to simulate and observe user paths within the system.	Validated navigation and data flows (e.g., job posting to review) to minimize rework during coding.	Role-separated screen flows, dashboard logic, and interaction details.
Document analysis	Reviewed existing documents (TA story drafts), data records (Backlog template), and course regulations.	Ideal for capturing non-functional requirements (e.g., the 3-course limit) and understanding existing data and processes.	Data field definitions, system processing rules, and non-functional constraints.

This phase utilized a **lightweight fact-finding strategy** focused on a "derive-sketch-refine" feedback loop to mitigate the risk of misunderstanding the core workflow. Due to the absence of direct client access, the team employed triangulation to ensure accuracy: using "Background reading" for hard constraints, and "Document analysis" to analyze existing records and regulations. This specifically allowed the capture of critical non-functional requirements (such as the 3-course limit for TA applications). Finally, "Observation" of the prototype was used to close the logic loop. This efficiently answered the "who, what, and how" of the system.

1.3 Iteration plan

The backlog is treated as a **living planning artefact**. After consolidating the stage-one evidence, we grouped stories into four delivery-oriented sprints. The sequencing principle was dependency first, then user value. Sprint 1 concentrates on the minimum runnable path because later operations, such as reminders, edit controls, exports and audit functions, only make sense once profiles, vacancies and applications already exist. Sprint 2 therefore wraps control and feedback features around that path; Sprint 3 adds efficiency and governance improvements; Sprint 4 focuses on hardening, acceptance testing and packaging. This ordering aims to minimise rework by stabilising the central workflow before optimisation begins.

Sprint	Focus	Candidate stories	Expected outcome
Sprint 1 (Weeks 3–4)	Validate the core recruitment path and confirm file-based data structures.	TA001–TA004, MO001–MO003, AD002.	Backlog + prototype validated; first runnable workflow skeleton and persistence design.

Sprint 2 (Weeks 5–6)	Add control, feedback and policy-related functions around the core path.	TA005–TA006, MO004, AD001, AD003.	Improved notifications, editable vacancies, workload/policy support and version 2 demo.
Sprint 3 (Weeks 7–8)	Improve efficiency, auditability and search convenience.	TA007–TA009, AD004–AD005.	Filtering, logs, abnormal-account handling and usability improvements.
Sprint 4 (Weeks 9–12)	Hardening, testing, documentation and final packaging.	Integration, regression tests, user manual, demo preparation.	Release-ready submission package and final acceptance testing support.

A story is considered done only when its acceptance criteria are checked, the relevant local text files can be created/read correctly, and the code/UI changes are merged through the team workflow. This keeps iteration planning tied to demonstrable evidence rather than only design intention. In addition, every sprint review should check three kinds of evidence together: the updated prototype or implemented screen flow, the underlying local file behaviour, and the associated GitHub activity showing collaborative delivery. This is important because the coursework assesses visible Agile management as well as software output. A story is not considered complete simply because a screen exists; it must also behave consistently with the data design and be integrated through the team workflow.

The plan is intentionally directional rather than rigid. The current roadmap defines stable sprint goals, but the exact story sequence can still be revisited after each review when technical findings or feedback change the team's former decision.

1.4 Prioritisation method

We used **MoSCoW prioritisation** and then checked each story against dependency, user value and technical risk. A story could only be labelled **Must have** if removing it would break the recruitment system or materially weaken the prototype display. This produced a backlog of **10 Must-have, 4 Should-have and 4 Could-have stories**. The key principle was that "Must have" meant "necessary". TA001-TA004 and MO001-MO003 were all treated as Must have because without them there is no dependable way to create applicant data, publish jobs, review applications or communicate decisions. AD002 was also promoted into the Must-have tier because it operationalises the administrator role in the brief and creates a management response when vacancies remain open for too long.

Priority	Decision rule	Examples from backlog
Must have	Required for a meaningful demo of the recruitment workflow or compliance-critical visibility.	TA001–TA005, MO001–MO003, AD001–AD002.
Should have	Important improvements, but a manual workaround still exists in the short term.	TA006, MO004, AD003, AD004.
Could have	Useful efficiency or hardening features that can be deferred without breaking the core loop.	TA007–TA009, AD005; AI matching deferred to later work.

By contrast, stories such as TA006, MO004, AD003 and AD004 were marked as Should have because they improve usability, traceability or operational efficiency, but the process can be omitted if time is limited. The Could-have tier contains features that are useful only after the core path is stable, such as extra search convenience or one-click account freezing. This kept the team from prioritising interface polish ahead

of workflow completion. It also created a transparent explanation for why AI matching remains outside the first implementation slice: its value is recognised, but it depends on trustworthy structured input and explainable logic.

1.5 Estimation method

To ensure a realistic and traceable delivery plan, the team utilized Relative Story Point Estimation (1, 2, 3, 5) rather than fixed hours. This approach prioritizes long-term stability by evaluating five core complexity drivers: screen breadth, validation complexity, local file I/O, cross-role dependency, and uncertainty.

The backlog is anchored by clear reference points to maintain consistency across the project. Two-point stories like TA005 and MO003 serve as baselines, extending existing flows with minimal data changes. Three-point stories, such as TA002 and MO001, involve moderate file handling or interface expansion. The five-point story, TA001, represents the system's richest data object and carries the highest initial uncertainty. This distribution—featuring one large epic, several medium-complexity tasks, and manageable two-point items—provides the team with the necessary flexibility to balance workloads without compromising sprint goals.

Scale	Typical effort	Example stories	Why that estimate is reasonable
1 pt	Very small	AD003 export filter option	Limited logic on top of existing data structures, low uncertainty.
2 pts	Small feature	TA005 application statement; MO003 status marking	Single workflow extension with clear acceptance rules and modest dependencies.
3 pts	Medium feature	TA002 CV upload; TA003 job overview; MO001 publish jobs	Multi-field interface behaviour plus file validation or structured write/read operations.
5 pts	Large / uncertain	TA001 comprehensive profile	Broad data model, repeated fields, stronger validation and more persistence logic.

Estimation serves as a tool for capacity planning rather than a rigid deadline. By moving away from intuition-based planning, the team can prevent development silos and mitigate blockers. Sprint packages combine interface work with backend validation, ensuring that a single blocked story does not halt overall progress. Furthermore, the process treats estimation as an ongoing mechanism. Under our re-estimation policy, any hidden complexities discovered—such as new persistence logic or cross-role synchronization—trigger a backlog refinement. This prevents scope creep from being buried within outdated point values.

Finally, we intentionally linked MoSCoW prioritization with story sizing. The "Must-have" first increment consists of medium-sized stories (2-3 points) rather than massive epics. This granularity makes the plan more resilient, allowing for easier rebalancing when lab feedback or technical conflicts arise. Overall, this stage-one framework transforms the initial brief into a traceable, role-balanced delivery plan. It provides a disciplined basis for implementation while remaining adaptable to the evolving technical constraints of the project, ensuring the team stays aligned with the core customer needs throughout the development lifecycle.

Overall, the stage-one outputs turn the brief into a traceable, realistic, and role-balanced delivery plan. The project now has a defined end-to-end business slice, a prototype-backed interaction model, a prioritized backlog, and an estimation approach that supports incremental delivery under clear technology constraints. This gives the team a solid basis for implementation without losing traceability or drifting from the main customer need.

2. Supporting material

The supporting material below is rebuilt only from the interview with our TA and feedback from our TA. We also did a little survey to our friends and teachers when we decided some important issues in our user stories.

Appendix A. Interview questions

ID	Target	Interview question	Requirement area validated
Q1	TA / AD	What is the realistic upper limit of concurrent TA appointments, and is a simple cap enough for the first release?	Workload rule and allocation visibility
Q2	TA / AD	If a TA is already working on other modules, what should the system show to decision-makers?	Existing-assignment visibility for MO/Admin
Q3	TA	Should a TA create an account independently, or should an administrator create it on the applicant's behalf?	Ownership of the registration flow
Q4	TA / AD	Should identity or eligibility checking happen during the application request instead of after account creation?	Approval workflow and priority
Q5	TA	Which pieces of applicant information can be merged into one structured personal-information form?	Story consolidation and data-entry design
Q6	TA	Should the CV be submitted as a PDF upload rather than as a separate link?	Document submission format
Q7	MO	Is timetable information available early enough to be mandatory, or should it remain optional?	Optional vs mandatory vacancy fields
Q8	MO	Is a fixed application deadline essential, or is it better for the MO to open and close recruitment manually?	Vacancy status control
Q9	Admin	What overview screens are needed to monitor TA supply, pending items and data export?	Admin reporting and dashboard scope
Q10	Prototype review	Which window names, buttons and page splits should be changed to make the TA journey clearer?	UI wording and interaction flow

Appendix B. TA feedback

Source	feedback	Interpretation	Requirement response in stage one
TA reviewer	A TA can realistically serve on up to three modules at the same time; there is no need to model a separate “overload” case, but existing assignments can be marked.	A simple operational rule is more useful than a complicated workload score.	Keep the maximum-three-modules rule and expose current participation to decision-makers.
TA reviewer	This is a recruitment system, so the TA should create the account independently instead of the administrator creating it.	Registration is applicant-led rather than admin-led.	Remove admin-created TA accounts from the core release and keep self-registration in scope.
TA reviewer	Verification can happen when the account or application is submitted rather than after the account is already created; this should have higher priority.	The workflow should avoid a redundant late approval step.	Move checking earlier in the journey and raise its implementation priority.
TA reviewer	Several information-entry stories can be merged; language score and similar items can be filled in together as part of basic information.	The backlog was too fragmented from the applicant point of view.	Consolidate multiple TA data-entry stories into one personal-information flow.
TA reviewer	The TA submits the CV itself, preferably as a PDF; a link is only useful for locating the stored file inside the system.	The real requirement is file upload, not link sharing.	Treat PDF CV upload as a core requirement and avoid a link-only design.
TA reviewer	Before the semester starts the MO may not know the full timetable, so that field should not be mandatory.	The system should not block vacancy creation with unavailable data.	Change timetable information from mandatory to optional.
TA reviewer	One lower-volume feature can be given lower priority because the number of courses is not always large.	Rare operational cases should not dominate the first release.	Downgrade the low-frequency feature from the core slice to a later increment.
TA reviewer	An application deadline is not very necessary; instead the MO can decide whether to close recruitment.	Manual vacancy status control is more practical than a rigid deadline rule.	Replace a hard deadline requirement with an MO-controlled open/close action.
Team follow-up	The TA user stories were rewritten according to the previous discussion, and the priority language was updated to follow the MoSCoW wording shown in the PPT.	Feedback directly changed both story wording and prioritisation style.	Align the backlog and report with Must / Should / Could style decisions.

Source	feedback	Interpretation	Requirement response in stage one
Team follow-up	The separate administrator approval step can be deleted.	The team agreed that an extra approval stage would add friction without enough value.	Remove the redundant admin-review step from the stage-one workflow.
Team follow-up	The administrator needs two main views: an all-TA dashboard with export and pending records, and an all-course dashboard showing TA allocation and highlighting understaffed modules.	The admin role is mainly about oversight, visibility and lightweight operations support.	Add dashboard / export / alert requirements to the admin scope, most likely as Should-have items.
Prototype review	The overall structure is acceptable, but the “Resume” page should be split into structured text entry plus PDF upload, renamed “Personal Information”, use “Save” instead of “Submit”, and remove the separate application-statement window.	Usability problems were concentrated in wording and unnecessary page separation, not in the whole architecture.	Refine the prototype and merge related stories around a clearer TA application journey.

Appendix C. Brief survey results

Issues	Support level	Resulting requirement response
Simplify TA information entry and combine it with PDF CV upload	High	Merge fragmented stories, keep one clearer personal-information flow, and retain PDF upload as core functionality.
Remove unnecessary approval or account-creation steps from the TA journey	High	Keep self-registration and remove the redundant admin approval step from the first increment.
Use MoSCoW wording and re-prioritise around the core recruitment path	Medium-High	Re-label stories with Must / Should / Could and protect the end-to-end recruitment slice first.
Make timetable optional and let MOs manually close recruitment instead of relying on a hard deadline	Medium	Model flexible vacancy status rules rather than over-constraining early data entry.
Give administrators overview dashboards, export support and visibility of under-staffed modules	Medium	Retain monitoring / reporting capability in scope, but place it behind the main applicant-application flow if time becomes tight.